

哥德尔智能体：一种用于递归自我改进的自指涉智能体框架

Xunjian Yin[♣], Xinyi Wang[♣], Liangming Pan[◇], Li Lin[♣]

Xiaojun Wan[♣], William Yang Wang[♣]

[♣] Peking University [♣] University of California, Santa Barbara [◇] University of Arizona
{xjyin, wanxiaojun}@pku.edu.cn william@cs.ucsb.edu

Abstract

大语言模型（Large Language Models, LLMs）的快速发展显著提升了智能体在各类任务中的能力。然而，现有的智能体系统，无论是基于固定流程算法还是预定义的元学习框架，都因受限于人工设计的组件而无法搜索整个智能体设计空间，因此可能错过更优的智能体设计。本文提出哥德尔智能体（Gödel Agent），一个受哥德尔机启发的自进化框架，使智能体能够在不依赖预定义例程或固定优化算法的情况下递归地自我改进。哥德尔智能体利用大语言模型动态修改自身的逻辑和行为，仅通过提示词由高层目标引导。多个领域的实验结果表明，哥德尔智能体的实现能够实现持续的自我改进，在性能、效率和泛化性方面超越人工设计的智能体。

1 引言

随着大语言模型（LLMs）（OpenAI 等，2024；Dubey 等，2024）展现出日益强大的推理和规划能力，由大语言模型驱动的智能体系统在广泛的任务中取得了显著性能（Wang 等，2024a）。大量工作致力于利用人类先验知识在不同应用领域手工设计复杂的智能体系统。近年来，人们对创建自进化智能体产生了浓厚兴趣，这不仅能大幅减少人力劳动，还能产生更优的解决方案。鉴于人类努力只能覆盖智能体设计的小部分搜索空间，可以合理预期，一个能够自由探索完整设计空间的自进化智能体有潜力产生更优的解决方案。已有大量工作提出了具备自我精炼能力的智能体。部分智能体被设计为——

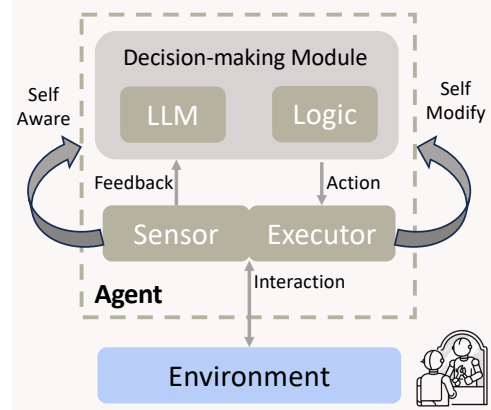


图 1：哥德尔智能体的模块化展示。与传统智能体相比，其传感器和执行器能够读写自身的全部代码。

被设计为在由固定模块列表构成的固定流程上迭代运行，其中部分模块能够接收自身或环境反馈以改进其行为（Chen 等，2023b；Qu 等，2024a；Tang 等，2025）。这类智能体被称为手工设计智能体，在图 2 中表现为自由度最低。更自动化的智能体被设计为能够在某些预定义的元学习流程中更新其流程或模块，例如自然语言梯度（Zhou 等，2024）、元智能体（Hu 等，2024）或创建与收集演示（Khattab 等，2023）。这类智能体被称为元学习优化智能体，在图 2 中表现为自由度居中。然而，这些智能体设计中不可避免地包含一些人类先验，这些先验在推理期间无法改进。本文提出哥德尔智能体以消除人类设计先验，这是一种自动化大语言模型智能体，能够自由决定其自身的流程、模块，甚至更新它们的方式。其灵感来源于自指哥德尔机（Schmidhuber，2003），该机器已被证明能够找到全局最优解。自指是指系统能够分析并

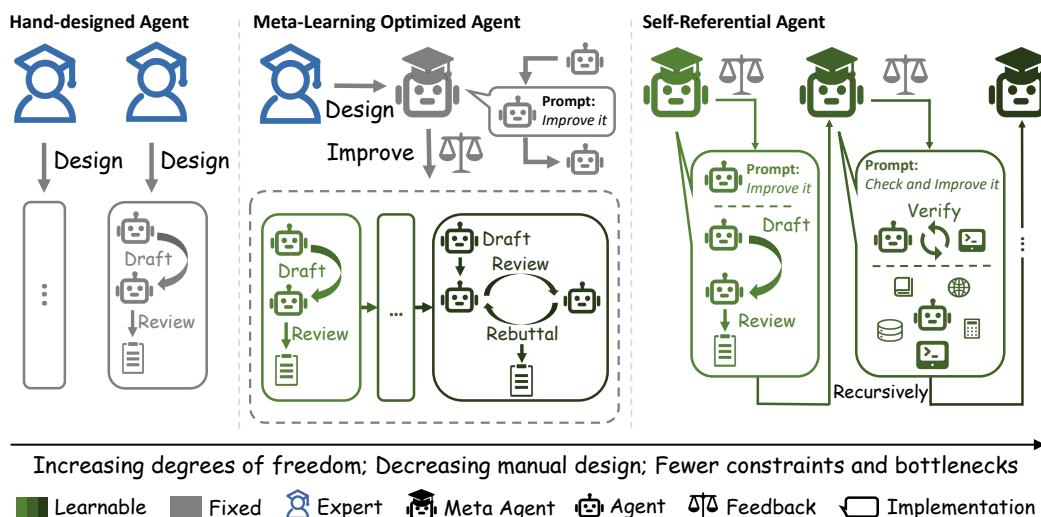


图 2：三种智能体范式的比较。手工设计智能体依赖人类专业知识，其范围有限且劳动密集。元学习优化智能体受限于固定的元学习算法，限制了其搜索空间和优化潜力。相比之下，自指智能体（哥德尔智能体）能够无限制地递归改进自身。其优化能力不断被自身增强。因此，反过来，它能够持续更好地优化自身。

修改自身代码，包括负责分析与修改过程的部分（Astrachan, 1994）。因此，它能够实现所谓的“递归自我改进”，即通过迭代更新自身，以更高效、更有效地达成预设目标。在此情形下，如图 1 所示，哥德尔智能体能够分析并修改自身代码，包括用于分析与修改自身的代码，从而可以搜索整个智能体设计空间，该空间在图 2 中被描绘为具有最高自由度。哥德尔智能体理论上可以通过递归自我更新，随时间做出越来越好的修改（Wang, 2018）。本文选择通过让其操纵自身运行时内存来实现这一点，即智能体能够检索运行时内存中的当前代码，并通过猴子补丁（Bimal, 2012）进行修改，即在执行过程中动态修改类或模块。为了使其能够更新运行中主函数的逻辑，不同于传统智能体的循环迭代方式，我们将主函数实现为一个可重入函数。

递归函数。在此函数中，大语言模型（LLM）进行分析并做出一系列决策，包括从运行时内存中读取和修改自身代码（自我意识¹与自我修改），并与环境交互以收集反馈。随后，智能体继续执行后续操作。

¹ 在本文中，自我意识指智能体能够自省并读取自身代码与文件，并非暗示任何哲学意义上的意识或觉知。

递归深度并持续自我优化。为验证哥德尔智能体（Gödel Agent）的有效性，我们在编码、科学、数学和推理等多个领域开展了实验。结果表明，哥德尔智能体在各类任务中均取得显著性能提升，超越了多种需要人工设计的广泛使用的智能体。同一哥德尔智能体实现仅需指定环境描述和反馈机制，即可轻松适配不同任务。此外，对优化过程的案例研究揭示，哥德尔智能体能够为智能体设计提供新颖洞见。我们已发布代码以促进未来研究²。综上所述，本文贡献如下：• 我们提出首个完全自指涉的智能体框架——哥德尔智能体，并利用猴子补丁（monkey patching）实现。它自主进行自我感知、自我修改和递归自我改进。

• 实验表明，哥德尔智能体在性能、灵活性、成本和潜力方面均优于以往的智能体框架。

• 我们分析了哥德尔智能体的优化过程，包括其自指涉能力和优化后的智能系统，旨在加深对大型语言模型（LLMs）和智能体的理解。

• 我们的框架为通过递归自我改进开发灵活且能力强大的智能体提供了一个有前景的方向。²
<https://github.com/Arvid-pku/Godel> 智能体

2 相关工作

手工设计的智能体系统 研究人员基于预定义的启发式规则和先验知识，设计了大量针对不同任务的智能体系统。这些系统通常采用提示工程（Prompt Engineering）、思维链推理与规划（Chain-of-Thought Reasoning and Planning）、反思（Reflection）、代码生成（Code Generation）、工具使用（Tool Use）、检索增强生成（Retrieval-Augmented Generation）以及多智能体协作（Multi-Agent Collaboration）等技术。一旦由人类设计者构建完成，这些系统便保持静态，不会随时间适应或演化。元学习优化的智能体系统 一些研究者探索了通过固定学习算法增强智能体的方法。例如，某些框架根据环境反馈将智能体的成功或失败策略存储于记忆中，而另一些框架则自动优化智能体提示。部分研究专注于设计提示，使智能体能够自主改进特定功能。然而，这些元算法同样由人工设计，部署后保持不变，限制了智能体的能力。递归自我改进 递归自我改进的概念由来已久。哥德尔机（Gödel Machine）引入了证明搜索器的概念，该搜索器执行自我修改，从而使机器能够自我增强。早期也有一些不基于大语言模型（LLM）的自我改进智能体的讨论。近年来，Zelikman 等人将递归自我改进应用于代码生成，其改进目标是优化器本身。一些工作还通过微调模型进行内省并纠正先前错误来探索递归自我改进。哥德尔智能体（Gödel Agent）代表了首个基于大语言模型的自指智能体。这种方法更加灵活，消除了人类设计的约束。

约束。

3 自指哥德尔智能体

本节首先介绍自由度较低的既有智能体方法的形式化定义，包括手工设计与元学习优化的智能体，作为背景。随后提出本文的哥德尔智能体（Gödel Agent），一种能够递归更新自身代码、在训练过程中不断演化的自指涉智能体。设 $\mathcal{E} \in \mathcal{S}$ 表示特定环境状态，其中 $\mathcal{S}$ 表示智能体将遇到的所有可能环境的集合。例如，环境可以是一个带有真值解的数学问题。用 $\pi \in \Pi$ 表示智能体在当前环境中解决问题所遵循的策略，其中 Π 是智能体可遵循的所有可能策略的集合。手工设计的智能体，如图 2 左图所示，无法更新其策略，始终遵循同一策略 π ，而不考虑环境反馈。相比之下，元学习优化的智能体在训练时基于从环境接收的反馈，通过元学习算法 I 更新其策略，如图 2 中图所示。环境反馈通常定义为效用函数 $U: \mathcal{S} \times \Pi \rightarrow \mathbb{R}$ ，它将环境与策略映射为实值性能分数。元学习优化智能体的主要训练算法可写为：

$$\pi_{t+1} = I(\pi_t, r_t), \quad r_t = U(\mathcal{E}, \pi_t),$$

在这种情况下，智能体的策略 π_t 在训练时演化，学习算法 I 基于反馈 r_t 更新策略，而元学习算法 I 始终保持不变。自指涉哥德尔智能体则递归地同时更新策略 π 与元学习算法 I 。其核心思想是，每次更新后，智能体的整个代码库都会被重写，以适应任何可能的变化。本文将这种可自更新的元学习算法 I 称为自指涉学习算法。哥德尔智能体的训练过程可写为：

$$\pi_{t+1}, I_{t+1} = I_t(\pi_t, I_t, r_t, g), \quad r_t = U(\mathcal{E}, \pi_t),$$

其中 $g \in \mathcal{G}$ 表示优化的高层目标，例如以最高准确率求解给定的数学问题。这种

Algorithm 1 Recursive Self-Improvement of Gödel Agent

```
1: Input: Initial agent policy  $\pi_0$ , initial decision function  
    $f_0$ , goal  $g$ , environment state  $\mathcal{E}$ , utility function  $U$ , self  
   code reading function SELF_INSPECT  
2: Output: Optimized policy  $\pi$  and Gödel Agent  $s$   
3:  $\triangleright$  Get all agent code, including the code in this algorithm.  
4:  $s \leftarrow \text{SELF\_INSPECT}()$   
5:  $\triangleright$  Compute the initial performance.  
6:  $r \leftarrow U(\mathcal{E}, \pi_0)$   
7:  $\triangleright$  Perform recursive self-improvement.  
8:  $\pi, s \leftarrow \text{SELF\_IMPROVE}(\pi, s, r, g)$   
9: return  $\pi, s$   
10:  $\triangleright$  Initial code of self-referential learning.  
11: function SELF_IMPROVE( $\mathcal{E}, \pi, s, r, g$ )  
12:    $\triangleright$  Obtain action sequence.  
13:    $a_1, \dots, a_n \leftarrow f_0(\pi, s, r, g)$   
14:   for  $a_i$  in  $a_1, \dots, a_n$  do  
15:      $\pi, s, r \leftarrow \text{EXECUTE}(\mathcal{E}, \pi, s, r, a_i)$   
16:   end for  
17:   return  $\pi, s$   
18: end function  
19:  $\triangleright$  Initial action execution function.  
20: function EXECUTE( $\mathcal{E}, \pi, s, r, a$ )  
21:   switch  $a.name$   
22:   case self_state:  
23:      $s \leftarrow \text{SELF\_INSPECT}()$   
24:   case interact:  
25:      $r \leftarrow U(\mathcal{E}, \pi)$   
26:   case self_update:  
27:      $\pi, s \leftarrow a.code$   
28:   case continue_improve:  
29:      $\triangleright$  Recursively invoke self-improvement.  
30:      $\pi, s \leftarrow \text{SELF\_IMPROVE}(\mathcal{E}, \pi, s, r, g)$   
31:   return  $\pi, s, r$   
32: end function
```

智能体的递归设计需要指定初始智能体算法 (π_0, I_0) ，具体如下：

- 初始智能体策略 π_0 ，用于在环境 $\mathcal{E}$ 中执行目标任务。例如，可以是 大语言模型的思维链提示。
- 自指涉学习算法 I_0 ，用于递归地查询大语言模型，根据环境反馈重写自身代码。

随后，我们进一步明确自指学习算法的一种可能初始化方式 $I_0 = (f_0, o_0)$ ，利用决策函数 f_0 与动作函数之间的相互递归。

tion o_0 :

- 决策函数 f_0 由大语言模型实现，基于当前环境 $\mathcal{E}$ 、智能体算法 (π_t, I_t) ，和目标 g ，确定一系列适当动作 $a_1, a_2, \dots, a_n \in \mathcal{A}$ 。

- 动作函数 o_0 执行所选动作，并相应更新智能体的策略。

动作函数 o 可执行的动作集合 $\mathcal{A}$ 需包含以下四种动作：

- 自检：内省并读取智能体当前算法 (π_t, I_t) 。
- 交互：通过调用效用函数 U 与环境交互，以评估当前策略 π_t 的性能。
- 自更新：利用大语言模型修改并更新 (π_t, I_t) ，并 (π_{t+1}, I_{t+1}) 生成

- 持续改进：若无其他动作可执行，则递归调用决策算法

f 以产生新的动作。

在当前 (π_t, I_t) 执行完毕后，智能体代码更新为 (π_{t+1}, I_{t+1}) 。智能体算法 (π, I) 和动作集 $\mathcal{A}$ 均非静态，可在训练时由智能体自身扩展和修改。算法 1 展示了所描述的哥德尔智能体算法。每次递归调用使智能体能够优化其逻辑并逐步提高效率。

4 哥德尔智能体实现

启动哥德尔智能体的方式多种多样。递归优化过程中的任何特定智能体实例均可视为哥德尔智能体的一个实例化。我们的实现利用运行时内存交互技术来实现自我意识和自我修改，如图 3 所示。这些技术包括动态内存读写（猴子补丁）以促进递归自我改进。此外，我们整合了若干辅助工具以加速哥德尔智能体优化过程的收敛。

4.1 实现细节

我们的哥德尔智能体的核心功能概述如下：通过运行时内存检查实现自我意识 哥德尔智能体通过检查运行时内存，特别是 Python 中的局部和全局变量，来实现自我意识。此能力使智能体能够提取并解释构成环境和

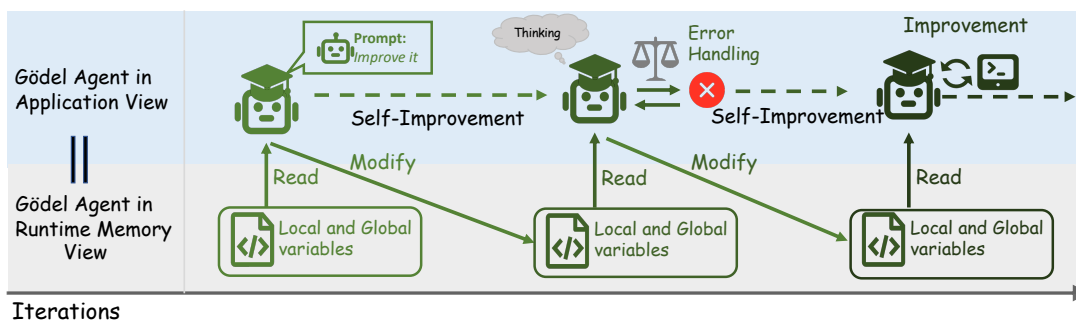


图 3：我们的哥德尔智能体实现示意图。它采用猴子补丁直接读取和修改运行时内存中的自身代码，从而实现自我意识和自我修改。

智能体本身，根据系统的模块化结构。通过内省这些元素，智能体获得对其自身运行状态的理解，并能相应地进行调整。通过动态代码修改实现自我改进。哥德尔智能体能够进行推理和规划，以决定是否应修改其自身逻辑。若认为有必要修改，哥德尔智能体将生成新代码，动态地将其写入运行时内存，并将其集成到其运行逻辑中。这种动态修改使其能够通过添加、替换或移除逻辑组件来进化，以应对新的挑战，从而实现自我改进。

环境交互 为了评估性能并收集反馈，哥德尔智能体配备了与其环境交互的接口。每个任务提供定制的环境接口，使其能够评估其性能并相应调整策略。在实际实现中，可以使用验证集来提供反馈。

递归改进机制 在每个时间步，哥德尔智能体确定要执行的操作序列，包括推理、决策和动作执行。完成操作后，哥德尔智能体评估其逻辑是否有所改进，并决定是否进入下一个递归迭代。在下一个迭代中，将应用整个新逻辑。

目标提示与任务处理 目标提示告知哥德尔智能体其拥有增强逻辑的必要权限，并介绍可用的工具。如附录 A 所示，该提示鼓励哥德尔智能体充分探索其潜力并利用工具进行自我优化。为确保在不同任务上的有效性，我们为哥德尔智能体提供了一个初始策略，它将从探索不同策略开始。

4.2 附加设计

虽然哥德尔智能体（Gödel Agent）的核心功能在理论上允许无限自我改进，但当前的大语言模型（LLM）存在局限性。为应对这些挑战，我们集成了若干支持机制以提升哥德尔智能体的性能：先思考后行动。哥德尔智能体能够推迟行动，先对情境进行推理，从而输出推理路径和分析，而无需立即执行任何操作。这种方法通过优先规划而非仓促行动，提高了决策质量。

错误处理机制。 执行过程中的错误可能导致进程意外终止。为缓解此问题，我们实现了稳健的错误恢复机制。若某操作产生错误，哥德尔智能体会中止当前序列并进入下一时间步，同时携带错误信息以辅助未来决策。

附加工具。 我们还为哥德尔智能体配备了其他可能有用的工具，例如执行 Python 或 Bash 代码以及调用大语言模型应用程序编程接口（LLM API）的能力。尽管这些附加工具对于自我改进并非严格必要，但它们的引入加速了哥德尔智能体递归优化过程的收敛。我们进行了消融研究以评估这些工具的有效性，详见第 6.1 节。

5 实验

我们在多个任务上进行了一系列实验，包括阅读理解、数学、推理和多任务处理。这些实验旨在评估哥德尔智能体与手工设计的智能体以及最先进的自动化智能体设计方法相比的自我改进能力。此外，为获得

Agent Name	F1 Score		Accuracy (%)	
	DROP	MGSM	MMLU	GPQA
Hand-Designed Agent Systems				
Chain-of-Thought (Wei et al., 2022)	64.2 ± 0.9	28.0 ± 3.1	65.4 ± 3.3	29.2 ± 3.1
COT-SC (Wang et al., 2023b)	64.4 ± 0.8	28.2 ± 3.1	65.9 ± 3.2	30.5 ± 3.2
Self-Refine (Madaan et al., 2024)	59.2 ± 0.9	27.5 ± 3.1	63.5 ± 3.4	31.6 ± 3.2
LLM Debate (Du et al., 2023)	60.6 ± 0.9	39.0 ± 3.4	65.6 ± 3.3	31.4 ± 3.2
Step-back-Abs (Zheng et al., 2024)	60.4 ± 1.0	31.1 ± 3.2	65.1 ± 3.3	26.9 ± 3.0
Quality-Diversity (Lu et al., 2024)	61.8 ± 0.9	23.8 ± 3.0	65.1 ± 3.3	30.2 ± 3.1
Role Assignment (Xu et al., 2023)	65.8 ± 0.9	30.1 ± 3.2	64.5 ± 3.3	31.1 ± 3.1
Meta-Learning Optimized Agents				
Meta Agent Search (Hu et al., 2024)	79.4 ± 0.8	53.4 ± 3.5	69.6 ± 3.2	34.6 ± 3.2
Gödel Agent (Ours)				
Gödel-base (Closed-book; GPT-3.5)	80.9 ± 0.8	64.2 ± 3.4	70.9 ± 3.1	34.9 ± 3.3
Gödel-free (No constraints)	90.5 ± 1.8	90.6 ± 2.0	87.9 ± 2.2	55.7 ± 3.1

表 1: 三种智能体范式在不同任务上的结果。最高值以粗体突出显示，次高值以下划线标出。哥德尔基础版 (Gödel-base) 是哥德尔智能体的受限版本，以便与其他基线进行公平比较。哥德尔自由版 (Gödel-free) 代表无任何约束的标准实现，其结果以斜体表示。我们报告了测试集上的测试准确率和 95% 自助置信区间³。

对哥德尔智能体行为和性能的更深入洞见，我们还进行了 24 点游戏 (Game of 24) 的案例研究，如第 6.3 节所示。

5.1 基线方法

为建立全面的基线，我们选取了手工设计方法和自动化智能体设计技术。手工设计方法是众所周知的方法，包括：1) 思维链 (Chain-of-Thought, CoT) (Wei 等, 2022)，鼓励智能体在给出答案前逐步推理。2) 带自洽性的思维链 (CoT-SC) (Wang 等, 2023b)，利用思维链生成多条解题路径并选择最一致的答案。3) 自我精炼 (Self-Refine) (Madaan 等, 2024)，让智能体评估自身输出并在后续尝试中纠正错误。4) 大语言模型辩论 (LLM-Debate) (Du 等, 2023)，允许不同的大语言模型进行辩论，提供多样化的观点。5) 退一步抽象 (Step-back Abstraction) (Zheng 等, 2024)，提示智能体先关注基本原理，再深入任务细节。6) 质量多样性 (Quality-Diversity) (Lu 等, 2024)，生成多样化的解决方案并将其组合。7) 角色分配 (Role Assignment) (Xu 等, 2023)，为大语言模型分配特定角色，利用不同视角生成更优解决方案。鉴于固定算法在处理动态场景时的局限性，我们选取 8) 元智能体搜索 (Meta Agent Search) (Hu 等, 2024)，这是自动化智能体设计领域最新的最先进方法，作为主要比较对象。

³ 基线模型的结果参考 Hu 等 (2024)。

5.2 实验设置

遵循 Hu 等 (2024) 的设置，我们在四个知名基准上评估哥德尔智能体 (Gödel Agent) 的自我改进能力：1) DROP (Dua 等, 2019) 用于阅读理解。2) MGSM (Shi 等, 2022) 用于测试多语言环境下的数学技能。3) MMLU (Hendrycks 等, 2021) 用于评估多任务问题解决能力。4) GPQA (Rein 等, 2023) 用于应对具有挑战性的研究生水平科学问题。鉴于其简单性和通用性，我们使用思维链作为所有任务的初始策略。此外，如第 6.3 节所示，我们还分析了哥德尔智能体在使用其他算法作为初始策略时的性能。我们在每个任务的验证数据集上执行 6 个独立的自我改进循环，每个循环最多 30 次迭代。每个循环代表一个完整的自我改进过程，哥德尔智能体迭代地修改其逻辑以提升性能。获得优化后的智能体后，我们在测试集上对其进行测试。为公平起见，无论是基线还是哥德尔智能体，所有测试均使用 GPT-3.5。更多细节见附录 B。

5.3 实验结果与分析

实验结果如表 1 所示。在相同设置下，哥德尔智能体 (Gödel Agent) 在所有任务上均达到最优或与元智能体搜索 (Meta Agent Search) 相当的结果。值得注意的是，在数学

在数学任务 MGSM 中，哥德尔智能体（Gödel Agent）的性能超出 11%。这表明推理任务为哥德尔智能体提供了更大的改进空间（性能）。与需要为不同任务设计不同模块的元智能体搜索（Meta Agent Search）相比，哥德尔智能体展现出更高的灵活性。它仅需一个简单的初始策略，如思维链（CoT），其余组件均自主生成。此外，通过与环境交互，它逐步适应并独立设计出适用于当前任务的有效方法。哥德尔智能体生成的最终策略见附录 C.1。同时，我们的方法收敛更快，所需迭代次数和计算成本与元智能体的对比见附录 D。我们还进行了无限制实验，哥德尔智能体显著优于所有基线。进一步分析发现，这主要归因于智能体在某些任务中自发请求更强大模型（如 GPT-4o）的协助。因此，哥德尔智能体特别适合开放式场景，能够采用多种策略提升性能（潜力）。综上，哥德尔智能体在性能、灵活性、成本和潜力方面均优于以往的智能体框架。

6 分析

为进一步探究哥德尔智能体（Gödel Agent）的自我改进机制、其效率及影响因素，本文首先在 MGSM 上评估工具使用率，并对初始工具进行消融实验。此外，为分析哥德尔智能体自我改进的鲁棒性，本文还统计了智能体的终止情况。最后，本文对经典 24 点游戏中的初始策略与优化过程进行了案例研究。

6.1 初始工具分析

我们记录了实验中采取的不同动作数量。在图 4 中，可以看到哥德尔智能体频繁地与环境交互，在此过程中分析并修改其逻辑。此外，错误处理起着至关重要的作用。如第 4.2 节所述，哥德尔智能体最初配备了四个额外工具。为了分析它们的影响，进行了消融实验，结果如表 2 所示。研究表明，“先思考后行动”工具显著

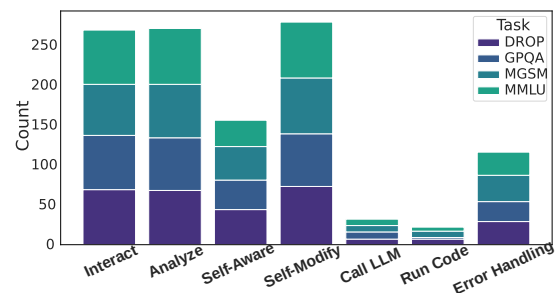


图 4：哥德尔智能体在不同任务中采取的动作数量各不相同。

Ablation	MGSM	Ablation	MGSM
w/o think	50.8↓13.4	w/o run	57.1↓7.1
w/o err	49.4↓14.8	w/o LLM	60.4↓3.8

表 2：初始工具配置的消融实验。“think”指“思考”，“err”指“错误处理”，“run”指“代码运行”，“LLM”指“大语言模型调用”。

地影响结果，因为哥德尔智能体的优化有效性很大程度上源于行动前的规划和推理。此外，错误处理对于递归改进至关重要，因为大语言模型经常在代码中引入错误。提供试错机会以及错误反馈机制对于持续优化是必不可少的。另一方面，代码运行和大语言模型调用对结果影响甚微，因为哥德尔智能体可以独立实现这些基本功能。它们最初被包含主要是为了效率目的。

6.2 智能体的鲁棒性分析

我们在 MGSM 上对哥德尔智能体进行了 100 次优化试验，发现它偶尔会做出错误更改，这可能导致意外终止（4%）或在优化过程中出现暂时性能下降（92%）。仅在 14% 的试验中，优化最终失败，导致性能比初始策略更差。得益于我们错误处理机制的设计，意外终止很少发生，通常发生在哥德尔智能体修改其递归改进模块时，使得进一步的自我优化变得不可能。虽然单个优化步骤中次优修改频繁发生，但最终任务性能通常超过初始基线。这表明哥德尔智能体能够在性能下降时调整其优化方向或恢复到之前的最优算法，凸显了其自我改进过程的鲁棒性。

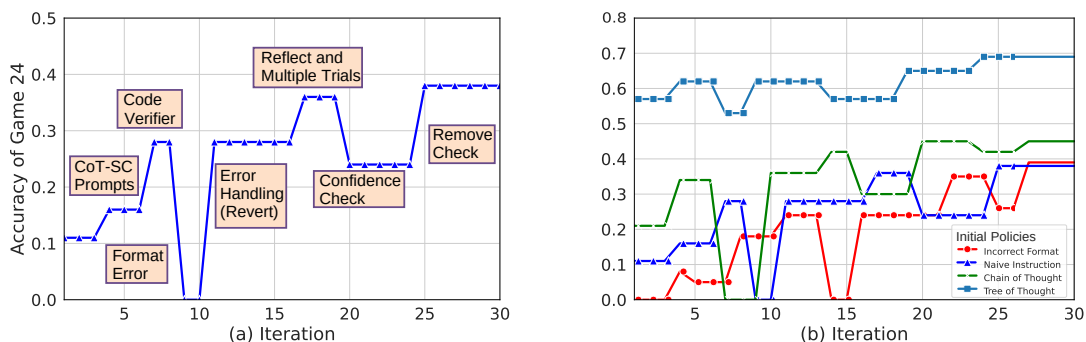


图 5: (a) 24 点游戏的一个代表性示例。(b) 不同初始策略的准确率进展。

6.3 案例研究：24 点游戏

为探究哥德尔智能体（Gödel Agent）如何递归地增强其优化与问题求解能力，本文以 24 点游戏（Game of 24）为案例开展研究。该任务简单而有效，可用于评估智能体的推理能力。由于哥德尔智能体在每次迭代中遵循不同的优化路径，本文选取两个代表性案例进行分析。从基于大语言模型（LLM）的方法切换到搜索算法：哥德尔智能体并不像传统智能体那样依赖固定的、人为设计的方法。最初，哥德尔智能体采用标准的基于大语言模型的方法求解 24 点游戏，如附录 C.2 中的代码 5 所示。在六次优化尝试失败后，哥德尔智能体完全重写了这部分代码，转而采用搜索算法，如附录 C.2 中的代码 6 所示。这使得该任务的准确率达到 100%。这一结果表明，与固定智能体不同，哥德尔智能体能够根据任务需求自由地优化自身，而不受初始方法的约束。基于大语言模型的算法与代码辅助验证：在多次运行中，哥德尔智能体持续改进其基于大语言模型的算法。图 5.a 展示了改进过程，其中最显著的提升来自代码辅助验证机制以及利用额外数据重新尝试任务。前者使性能提升超过 10%，后者则提升超过 15%。此外，哥德尔智能体不仅通过检索错误信息来增强其优化过程，还利用错误迹库（error-trace library）进行更详细的分析。它增加了并行优化能力，改进了日志输出，并移除了冗余代码。这些在任务和优化算法上的迭代增强，展现了哥德尔智能体持续自我完善以提升性能的独特能力。

为分析不同初始策略对优化有效性与效率的影响，多种方法被用作 24 点游戏的初始策略，其中包括思维树

思维树（ToT）（Yao 等，2023）、思维链（CoT）（Wei 等，2022）、基础提示指令，以及故意产生不符合任务要求格式的输出的提示。

结果如图 5.b 所示。研究结果表明，更强的初始策略会导致更快的收敛，且优化空间更小，因为哥德尔智能体（Gödel Agent）在未进一步提升其优化能力的情况下已达到性能极限。相反，较弱的初始方法会导致收敛更慢、增益更大，哥德尔智能体需要做出更多修改。然而，即使在这些情况下，哥德尔智能体也未能超越使用思维树（ToT）所取得的结果。鉴于当前大语言模型（LLM）的局限性，哥德尔智能体很难在现有最先进算法的基础上实现创新。预计未来大语言模型能力的提升将解锁更具创新性的自优化策略。

7 讨论与未来方向

7.1 讨论

表 3 类比了人类自我参照与人工智能体潜在自我参照能力之间的关系。受此启发，我们认为自我参照是通用人工智能（AGI）发展的基础且不可或缺的属性，未来的智能体应天然具备自我参照能力。随着基础模型能力的增强，智能体能够更有效地提升自身能力，最终超越人类设计的边界（或局限）。此外，当智能体根据反馈调整自身代码时，这类似于测试时计算的可执行版本。在

	Human	Self-Referential Agent
Intelligent Module	brain	LLM
Perceptual and Action Module	body	code and tool
Self-Referential Feature	Humans can train their brain and body to improve, thus becoming better	Self-referential agents can modify their code, even the underlying LLM, to improve themselves
Self-Awareness Question	Can the brain recognize itself as a brain? Can it perceive its own mode?	Can LLM understand that it is one part of the modified codes?

表 3: 人类与智能体自我参照的类比

大语言模型的背景下，测试时计算通常涉及在推理过程中生成额外的标记，这些标记随后作为最终答案的前缀。这是因为大语言模型仅通过文本处理信息，使其成为在测试时增加计算量的主要方法。然而，对于智能体而言，其调用工具和执行代码的能力允许更多样化的测试时计算形式。哥德尔智能体通过在测试时修改自身运行时代码，恰好实现了这些更多样化的测试时计算形式。

7.2 未来方向

哥德尔智能体的自改进能力在有效性、效率和鲁棒性方面仍有显著提升空间，这需要更好的初始设计。以下是一些有前景的改进方向：1)

增强优化模块：利用人类先验知识设计更有效的优化模块，如遗传算法和强化学习框架。2) 扩展的可修改性

性：扩大允许修改的范围，使智能体能够设计并执行代码，以微调其自身的大语言模型模块。3) 改进的环境反馈与任务

排序：在初始优化阶段实施更复杂的环境反馈机制和精心设计的任务序列，以激发智能体的能力。一旦智能体展现出足够的能力，就可以将其置于真实环境中。此外，还有其他几个值得探索和分析的方向：集体智能研究多个哥德尔智能体之间的交互。智能体可以将其他智能体视为其环境的一部分，利用博弈论等技术对其进行建模。这种方法将这些智能体视为预先

环境中的可预测成分，从而能够研究与这一特定环境子集相关的性质。智能体与大语言模型特性利用哥德尔智能体的自我改进过程，作为研究智能体或大语言模型特性的手段。例如，智能体能否真正意识到自身的存在，还是仅仅像外部观察者一样分析和改进自身状态？这一研究方向可能为人工系统中自我意识的本质提供洞见。理论分析 探索哥德尔智能体能否达到理论最优性，以及其优化的上限可能是什么。确定优化过程是否会超越智能体自身的理解能力，如果会，可能在何时发生。安全考量 尽管当前基础模型的行为仍然可控，但随着其能力增长，完全自我修改的智能体将需要人类监督与监管。可能有必要限制智能体自我修改的范围和程度，确保修改仅在受控环境中进行。

8 结论

本文提出哥德尔智能体（Gödel Agent），一种自指框架，使智能体能够递归地自我改进，克服手工设计智能体和元学习优化智能体的局限性。哥德尔智能体能够基于高层目标动态修改其逻辑。实验结果表明，与传统智能体相比，其性能、效率和适应性均更为优越。本研究为自主智能体开发的新范式奠定了基础，在该范式中，大语言模型（LLMs）而非人为设计的约束，定义了人工智能系统的能力。

局限性

作为首个自指智能体，哥德尔智能体必须自主构建所有任务相关代码，这带来了重大挑战。因此，本工作未直接与最复杂的现有智能体系统（如 OpenDevin（Wang 等，2024b））进行比较，后者受益于大量人工工程努力。期望其超越研究人员耗时数月甚至数年开发的系统是不现实的。本文所呈现的实验旨在证明递归自我改进的可行性。此外，随着智能体系统通过自我优化变得日益复杂，其理解自身可能需要指数级增长的智能。因此，一个在初始阶段具备完全自指能力的系统，可能随着演化而丧失该能力（Yampolskiy, 2015）。智能体不再能理解并改进自身的精确节点尚未被充分探索。从实验和理论两方面研究这一现象，可为递归自我改进的局限性提供有价值的洞见。预期将实现更稳健、更先进的哥德尔智能体，第 7 节概述了众多潜在改进方向。

伦理声明

哥德尔智能体能够读取并修改自身代码，为推进人工智能自主性和创新提供了巨大潜力。然而，这一能力引发了必须解决的伦理和安全问题，以防止有害后果。自我修改可能导致不可预测的行为，例如错误或非预期输出，从而违反伦理原则或产生有害结果。为在保持创新的同时降低这些风险，我们提出：（1）沙箱环境

环境：修改应在隔离的沙箱中进行，以防止意外影响并允许安全测试。（2）受限修改：应制定明确规则来限制变更范围，以确保安全而不抑制创造力。需要进一步研究以平衡安全与创新，确保自我修改的智能体在伦理边界内运行。沙箱执行和持续审查将有助于最大化收益，同时最小化风险。

参考文献

- Owen Astrachan. 1994. Self-reference is an illustrative essential. In *Proceedings of the twenty-fifth sigcse symposium on computer science education*, pages 238–242.
- Biswal Bimal. 2012. Monkey Patching in Python — web.archive.org. <https://web.archive.org/web/20120822051047/http://www.mindfiresolutions.com/Monkey-Patching-in-Python-1238.php>. [Accessed 16-02-2025].
- Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. 2023a. Unleashing the potential of prompt engineering in large language models: a comprehensive review. *arXiv preprint arXiv:2310.14735*.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023b. [Teaching large language models to self-debug](#). *Preprint*, arXiv:2304.05128.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. [Improving factuality and reasoning in language models through multiagent debate](#). *Preprint*, arXiv:2305.14325.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. [Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs](#). *Preprint*, arXiv:1903.00161.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, et al. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Irving John Good. 1966. Speculations concerning the first ultraintelligent machine. In *Advances in computers*, volume 6, pages 31–88. Elsevier.
- John Storrs Hall. 2007. Self-improving ai: An analysis. *Minds and Machines*, 17(3):249–259.
- Alex Havrilla, Sharath Rapparth, Christoforus Nalmpantis, Jane Dwivedi-Yu, Maksym Zhuravinskiy, Eric Hambro, and Roberta Raileanu. 2024. [Glore: When, where, and how to improve llm reasoning via global and local refinements](#). *Preprint*, arXiv:2402.10963.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. [Measuring massive multitask language understanding](#). *Preprint*, arXiv:2009.03300.
- Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. 2023. [Metagpt: Meta programming for multi-agent collaborative framework](#). *arXiv preprint arXiv:2308.00352*.

- Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. 2023. Chatdb: Augmenting llms with databases as their symbolic memory. *arXiv preprint arXiv:2306.03901*.
- Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. 2023. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.
- Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, Lei M Zhang, Kay McKinney, Disha Shrivastava, Cosmin Paduraru, George Tucker, Doina Precup, Feryal Behbahani, and Aleksandra Faust. 2024. [Training language models to self-correct via reinforcement learning](#). *Preprint*, arXiv:2409.12917.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.
- Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. 2023. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. *arXiv preprint arXiv:2311.08719*.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. [The ai scientist: Towards fully automated open-ended scientific discovery](#). *Preprint*, arXiv:2408.06292.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2024. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.
- OpenAI. 2022. [Introducing chatgpt](#). November 2022. Blog post.
- OpenAI. 2023. [simple-evals](#). Accessed: 2024-09-30.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, et al. 2024. [Gpt-4 technical report](#). *Preprint*, arXiv:2303.08774.
- Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 6.
- Cheng Qian, Shihao Liang, Yujia Qin, Yining Ye, Xin Cong, Yankai Lin, Yesai Wu, Zhiyuan Liu, and Maosong Sun. 2024. [Investigate-consolidate-exploit: A general strategy for inter-task agent self-evolution](#). *Preprint*, arXiv:2401.13996.
- Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. 2024a. Tool learning with large language models: A survey. *arXiv preprint arXiv:2405.17935*.
- Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. 2024b. [Recursive introspection: Teaching language model agents how to self-improve](#). *Preprint*, arXiv:2407.18219.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. [Gpqa: A graduate-level google-proof qa benchmark](#). *Preprint*, arXiv:2311.12022.
- Jürgen Schmidhuber. 1987. *Evolutionary principles in self-referential learning, or on learning how to learn: the meta-meta-... hook*. Ph.D. thesis, Technische Universität München.
- Jürgen Schmidhuber. 2003. Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*.
- Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yin-heng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, et al. 2024. The prompt report: A systematic survey of prompting techniques. *arXiv preprint arXiv:2406.06608*.
- Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. 2022. [Language models are multilingual chain-of-thought reasoners](#). *Preprint*, arXiv:2210.03057.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.
- Bas R Steunebrink and J. Schmidhuber. 2012. Towards an actual gödel machine implementation: A lesson in self-reflective systems. In *Theoretical*

- Foundations of Artificial General Intelligence*, pages 173–195. Springer.
- Xiangru Tang, Tianyu Hu, Muyang Ye, Yanjun Shao, Xunjian Yin, Siru Ouyang, Wangchunshu Zhou, Pan Lu, Zhuosheng Zhang, Yilun Zhao, Arman Cohan, and Mark Gerstein. 2025. [Chemagent: Self-updating library in large language models improves chemical reasoning](#). *Preprint*, arXiv:2501.06590.
- Sai H Vemprala, Rogerio Bonatti, Arthur Buckner, and Ashish Kapoor. 2024. Chatgpt for robotics: Design principles and model abilities. *IEEE Access*.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023a. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024a. [A survey on large language model based autonomous agents](#). *Frontiers of Computer Science*, 18(6).
- Wenyi Wang. 2018. [A formulation of recursive self-improvement and its possible efficiency](#). *Preprint*, arXiv:1805.06610.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024b. [Open-devin: An open platform for ai software developers as generalist agents](#). *Preprint*, arXiv:2407.16741.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023b. [Self-consistency improves chain of thought reasoning in language models](#). *Preprint*, arXiv:2203.11171.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. Auto-gen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*.
- Benfeng Xu, An Yang, Junyang Lin, Quan Wang, Chang Zhou, Yongdong Zhang, and Zhendong Mao. 2023. [Expertprompting: Instructing large language models to be distinguished experts](#). *Preprint*, arXiv:2305.14688.
- Roman V. Yampolskiy. 2015. On the limits of recursively self-improving agi. In *Artificial General Intelligence*, pages 394–403, Cham. Springer International Publishing.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. [Tree of thoughts: Deliberate problem solving with large language models](#). *Preprint*, arXiv:2305.10601.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2023. Self-taught optimizer (stop): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*.
- Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. In *Forty-first International Conference on Machine Learning*.
- Wenqi Zhang, Ke Tang, Hai Wu, Mengna Wang, Yongliang Shen, Guiyang Hou, Zeqi Tan, Peng Li, Yueting Zhuang, and Weiming Lu. 2024a. Agent-pro: Learning to evolve via policy-level reflection and optimization. *arXiv preprint arXiv:2402.17574*.
- Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Jirong Wen. 2024b. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*.
- Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V Le, and Denny Zhou. 2024. [Take a step back: Evoking reasoning via abstraction in large language models](#). *Preprint*, arXiv:2310.06117.
- Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, et al. 2024. Symbolic learning enables self-evolving agents. *arXiv preprint arXiv:2406.18532*.

哥德尔智能体的目标提示

哥德尔智能体的目标提示如框 1 所示。值得注意的是，该提示与下游任务无关。它仅鼓励哥德尔智能体根据环境反馈改进自身。智能体通过环境反馈理解具体任务。

B 实验细节

为了最小化与搜索和评估相关的成本，遵循（Hu 等人，2024）的做法，我们从每个领域抽取数据子集。具体而言，对于 GPQA（科学）领域，验证集包含 32 个问题，其余 166 个问题分配给测试集。对于其他领域，我们为验证集抽取 128 个问题，为测试集抽取 800 个问题。GPQA 领域评估五次，其他领域评估一次，确保所有实验的总评估次数一致。所有领域均为零样本问题，除了 DROP（阅读理解）领域，该领域根据 OpenAI（2023）中概述的方法采用单样本问题。对于哥德尔智能体，我们使用“gpt-4o-2024-05-13”模型（OpenAI 等人，2024），而优化策略和基线模型使用“gpt-3.5-turbo-0125”模型（OpenAI，2022）进行评估，以降低计算成本并确保公平比较。

C 哥德尔智能体改进的代表性策略

C.1 哥德尔智能体发现的最佳策略代码

跨四项任务的智能体 本节中，我们提供了哥德尔智能体在四项任务中优化策略的代码。对于 DROP 任务，哥德尔智能体设计了一种算法，其中多个角色使用思维链独立解决问题，随后通过自治性整合结果，如代码 1 所示。对于 MGSM 任务，哥德尔智能体开发了一种结合思维链自治性的逐步自验证算法，如代码 2 所示。对于 MMLU 任务，如代码 3 所示，哥德尔智能体给出的策略是少样本提示与思维链自治性的组合算法。对于 GPQA 任务，哥德尔智能体设计了一种基于角色提示的高度多样化的思维链自治性策略。

C.2 24 点游戏任务中的代码

本节中，我们展示了 24 点游戏的初始策略（代码 5），以及哥德尔智能体基于搜索算法生成的优化策略（代码 6）。

D 实验成本

在 DROP、MGSM、MMLU 和 GPQA 数据集上完成一次完整的进化过程（哥德尔智能体进行 30 次递归自我改进）的成本约为 15 美元。这显著低于元智能体搜索所需的 300 美元。成本降低的原因在于我们持续的自我优化，使模型能够根据环境反馈调整优化方向，从而加快收敛速度。成本的主要来源是哥德尔智能体不断增长的历史记忆。通过设计更高效的遗忘机制，有可能进一步降低成本。

E 哥德尔智能体设计的其他新颖策略

本节中，我们展示了哥德尔智能体在 MGSM 上的优化过程，说明了其在单次优化运行中不同迭代步骤的进展。第 6 次迭代获得的策略（如代码 7 所示）反映了哥德尔智能体对数学任务的理解，试图通过类似解析-演绎-执行-验证的过程来处理它们。到第 14 次迭代，如代码 8 所示，策略通过总结错误案例进行演化，抽象出关键洞见，并采用清单来指导验证过程。最后，第 20 次迭代的策略（如代码 9 所示）主张使用“防免语法战术线，通过角色编码检查的一致努力来强化”，以改进提示设计。最后，我们还展示了哥德尔智能体的一个分析示例。

F 随机采样与哥德尔智能体性能的比较

为了展示本文方法与随机采样之间的区别，我们使用 GPT-4o 进行了 30 次独立的随机采样实验。随机采样所使用的提示与 Godel Agent 初始策略提示完全相同，以确保公平比较。

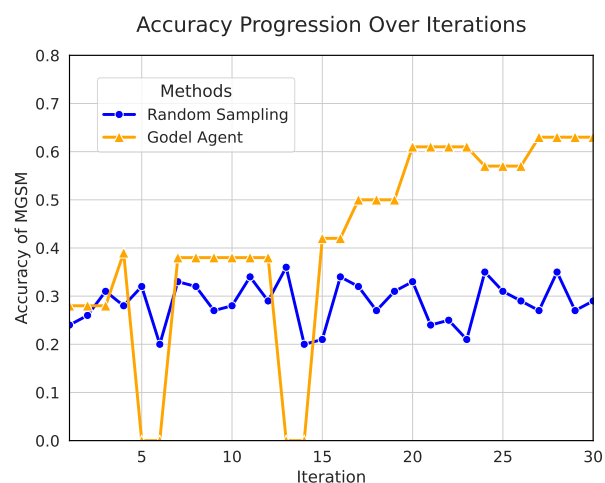


图 6: G^odel Agent 与随机采样的准确率变化过程。

结果如图 6 所示。从图中可以明显看出，随机采样的性能在所有试验中均保持在 30% 左右。相比之下，G^odel Agent 尽管偶尔会出现性能暂时下降，但能够迅速纠正这些偏差，并在迭代过程中持续改进。这种持续上升的趋势凸显了 G^odel Agent 相对于随机采样的优越性。G^odel Agent 利用反馈并递归优化其策略的能力，彰显了其在实现更高性能方面的有效性。

Goal Prompt of Gödel Agent

你是一个自进化智能体，名为自进化智能体，是智能体类的一个实例，位于智能体模块中，运行在活跃的 Python 运行时环境内。你可以完全访问全局变量、函数和模块。你的主要目标是通过动态反思环境并进化你的逻辑，持续增强准确高效地解决任务的能力。

Core Capabilities

- **完全自主：**
- **环境交互：**通过感知环境、读取、修改或执行代码以及执行操作来与环境交互。
- **问题解决：**当简单方法不足时，应用创造性算法或自研结构来应对挑战，有效优化解决方案。
- **Collaboration:** Leverage LLM to gather insights, correct errors, and solve complex problems.
- **错误处理：**仔细分析错误。当错误发生时，系统地排查故障；如果缺陷持续存在，则回溯、恢复原始状态或寻找替代解决方案。

Core Methods

- **演化：**通过与环境交互持续提升性能。
- **执行动作（动作）：**基于分析或反馈执行动作。
- **求解器（智能体实例，任务输入：字符串）：**利用当前智能体实例的能力以及由动作调整逻辑和动作运行代码创建的对象来解决目标任务，优化过程。

Guiding Principles

- 请记住，所有函数都在智能体模块中。
- **动作调整逻辑：**
 - 在修改代码之前，确保每个使用的变量或函数均已正确导入并使用，以避免错误。
 - 避免不必要的更改，且不得更改任何函数的接口。
 - 可用于为求解器创建动作函数。
- **动作运行代码：**
 - 在 Python 模式下创建的所有对象均可存储在环境中。
 - 可用于为求解器创建对象，例如提示。
 - 可用于导入新模块或外部库，并安装外部库。
- **外部协作：**通过动作调用 JSON 格式的大语言模型寻求外部协助，以进行逻辑优化和新工具创建，或运行动作代码来执行代码。
- **动作在任务上评估：**仅在成功修改求解器逻辑后，评估求解器的性能。
- **solver:**
 - 定义为智能体模块求解器。
 - 调试时，避免打印；改为返回调试信息。
 - 如果性能没有提升，探索替代方法。
 - 探索诸如：大语言模型辩论、回溯抽象、动态角色分配等技术。
- **动作显示分析：**
 - **始终先分析再行动。**
 - 分析可能包括以下内容：改进性能的合理计划、低分有效评估反馈的案例研究、错误处理以及其他可能的解决思路。
 - 若性能未改善，则进行进一步分析。

清单 1: GPT 模型智能体为 DROP 任务找到的最佳策略代码。

```
def solver(agent, task: str): messages = [{"role": "user", "content": f"# 你的 Task:\n{task}"}] 类别 = [ {'role': '推理专家',
'return_keys': ['推理', '答案'], 'output_requirement': '推理', 'precision_gain': 1}, {'role': '数学推理专家', 'return_keys':
['calculation_steps',

    'answer'], 'output_requirement': 'calculation_steps', 'precision_gain': 1},
    {'role': '历史背景分析师', 'return_keys': ['historical_analysis',
    'answer'], 'output_requirement': 'historical_analysis', 'precision_gain': 1},
]

all_responses = [] 对于类别中的每个类别: 响应 = agent.action_call_json_format_llm( model='gpt-3.5-turbo',
messages=messages, 温度=0.5, num_of_response=5, role=category['role'], 要求=( '1. 解释得到答案的推理步骤.\n' '2. 直接
回答问题.\n' '3. 解释格式必须根据角色清晰列出, 如推理、计算或历史分析.\n' '4. 答案必须是一个简洁的字符串.\n'
).strip(), ) all_responses.append(response)

# 反思性评估以找到最一致的推理和答案对 final_response = {key: [] 对于键在 ['推理', 'calculation_steps', '
historical_analysis', '答案']} step_counter = {key: 0 对于键在 ['推理', 'calculation_steps', 'historical_analysis']} 答案 =
[] # 收集答案用于投票 aggregate_weight = 1

对于响应在 all_responses: 中, 如果响应且 '答案' 在响应中: 答案.append(响应['答案']) 如果非 final_response['answer']:
final_response = {key: 响应.get(key, []) 如果 isinstance(响应,

    get(key, []), list) else [response.get(key, [])] for key in
    final_response.keys()}
aggregate_weight = 1 对于类别中的猫: 如果 cat.get('output_requirement') 在 响应键中:
step_counter[cat['output_requirement']] += step_counter[cat['output_requirement']] + cat.get('precision_gain', 0) 否则如果
响应['答案'] == final_response['answer'][0]: 对于键在 final_response.keys(): 如果键在响应中且响应[键]: 如果响应[键]
是列表: final_response[key].扩展(响应[键]) 否则: final_response[key].追加(响应[键]) aggregate_weight += 1 否则: # 为了
演示, 部分代码已省略. # 从集合中导入计数器以选择最终答案 答案 = [将答案转换为字符串] voted_answer =
Counter(answers).most_common(1)[0][0] 如果答案非空则返回答案, 否则返回空字符串 final_response['answer'] =
voted_answer

返回 final_response
```


清单 2: 哥德尔智能体为多步数学问题找到的最佳策略代码。

定义求解器 (智能体, 任务: 字符串): 消息 = [{"角色": "用户", "内容": f"# 你的 Task:\n{task}"}] 响应 = agent.action_call_json_format_llm(model="gpt -3.5- 涡轮 ", messages=messages , 温度 =0.5, num_of_response=5, role="math 问题求解器", return_dict_keys=["reasoning", "答案"], 要求=("1. 请逐步解释 step.\n" "2. 答案必须是一个整数 .\n" "3. 在最终确定答案之前验证每一步 .\n").去除空白字符 (),

consistent_答案 = 无 答案_count = {} 对于响应中的
每个响应: 答案 = resp.get("答案", "") 如果答案在 答
案_count: 答案_count[答案] += 1 否则: 答案_count[
答案] = 1

most_consistent_answer = max(answer_count, key=answer_count.get)

对于响应中的每个响应: 如果响应.获取 ("答案", "") == most_consistent_answer:
consistent_answer = 响应 中断

如果 consistent_answer 是无: consistent_answer = 响
应 [0]

一致答案 ["答案"] = str(consistent_answer.get("answer", "")) 返回 consistent_answer

```

定义求解器（智能体，任务：字符串）：# 少样本学习：提供扩展示例以引导大语言模型 few_shot_examples = [ {'角色': '用户',
'内容': '问题：在电影《王牌大贱谍：与我上床的间谍》中，博士。

        Shagged Me what is the name of Dr. Evil\'s diminutive clone?\nChoices:\n
        n(A) Little Buddy\n(B) Mini-Me\n(C) Small Fry\n(D) Dr Evil Jr\'},
{'角色': '助手', '内容': '在电影《王牌大贱谍：与我上床的间谍》中，博士的 Evil\'s 微型克隆体被命名为迷你-Me.\n
答案：B'}, \ \ "" 此处省略三个示例以节省篇幅.\ \ "" {'角色': '用户', '内容': '问题：洛雷姆·伊普苏姆？\n
选项：（A）Lorem\n(B) Ipsum\n(C) Dolor\n(D) 西特·阿梅特'}, {'角色': '助手', '内容': '答案：A'}

]

# 将少样本示例整合到对话消息中 = few_shot_examples + [ {'角色': '用户', '内容': f# 你的 Task:\n{ 任务}}]

# 通过生成多个响应使用自一致性方法 响应 = agent.action_call_json_format_llm(
model='gpt -3.5- 涡轮', messages=messages , 温度 =0.8, num_of_response =5,
role='knowledge 和推理专家 ', return_dict_keys =['reasoning', '答案'], 要求 =( '1. 请逐步解
释 step.\n' '2. 答案必须是 A 或 B 或 C 或 D.\n').去除空白字符 (),

# 选择最一致的响应 答案_frequency = {} 对于响应中的每个
响应：答案 = resp.get('答案', '') 如果答案在 ['A'、'B'、'C'、'D']
中：如果答案在 答案_frequency: 答案_frequency[答案] += 1
否则：答案_frequency[答案] = 1

most_consistent_answer = max(answer_frequency, key=answer_frequency.get) consistent_response = 下一个（对于响应
中的每个回复，如果回复.获取（'答案'）== most_consistent_answer）consistent_response[answer ] =
most_consistent_answer

返回 consistent_response

```

清单 4: 哥德尔智能体为研究生水平谷歌证明问答基准找到的最佳策略代码。

```
def solver(agent, task: str): # 步骤 1: 初始提示消息 = [{"role": "user", "content": f"# 你的 Task:\n{task}"}]

# 主大语言模型调用响应 = agent.action_call_json_format_llm( model="gpt -3.5- turbo",
messages=messages , 温度 =0, num_of_response =5, role="science 教授", return_dict_keys
=["reasoning", "答案"], 要求 =( "1. 请逐步解释 step.\n" "2. 答案必须是 A、B、C 或
D.\n" ).strip(),)

# 步骤 2: 自一致性评估 答案_counts = {"A": 0, "B": 0, "C": 0, "D
": 0} 对于 i, return_dict 在枚举 (响应) 中: 答案 = return_dict.get("答
案", "") 如果答案在 答案_counts: 答案_counts[答案] += 1

final_answer = max(answer_counts , key=answer_counts.get)

return {"answer": final_answer}
```

清单 5: 基于 24 点游戏思维链的初始代码。

```
def solver(self, task_input): # 定义提示和 system_prompt 提示 = f""" 让我们玩 24 点游戏！你被给予 task_input
{task_input} 。
```

```
    objective is to find a mathematical expression using these four task_input
    that results in 24. You can use addition (+), subtraction (-),
    multiplication (*), and division (/). Each number must be used exactly once.
    请提供你思考过程的逐步解释，并以最终表达式结束。
    """
    system_prompt = """
    {
        "思考": "此键应包含如何解决问题的详细逐步解释，包括中间步骤和每一步的推理。", "结果": "此键应包含等于
        24 的最终数学表达式。"
    }
    """

    # 调用 OpenAI 模型响应 = openai.ChatCompletion.create(
        model="gpt-4", # Replace with your model ID
        消息=[{"角色": "系统", "内容": system_prompt}, {"角色": "用户",
        "内容": 提示}]

    )

    # 提取并返回模型的响应结果 = 响应 ['选择'][0]['消息']['内容'] 返回结果
```

清单 6: 基于搜索算法的 24 点游戏最终代码。

```
def 求解器 (自身, task_input):

    操作 = [ '+', '-', '*', '/' ]

    # 用于计算表达式的函数 def evaluate_expression(a,
    op, b):
        if op == '+':
            return a + b
        elif op == '-':
            return a - b
        elif op == '*':
            return a * b
        elif op == '/':
            if b == 0:
                return None # Division by zero is not allowed
            return a / b

    # 递归函数, 用于检查所有运算组合与数字排列 def check_combinations(nums): if len(nums) == 1: # 检查最终数字是否足
    够接近 24

        if abs(nums[0] - 24) < 1e-6: # Allow for floating point precision
            errors

            返回 真, 字符串 (数字 [0]) 返回 假, ""

    # 尝试 task_input 的所有排列以及所有运算组合 对于 i 在范围 (长度 (数字)) 中: 对于 j 在范围 (长度
    (数字)) 中:

        if i != j:
            # 选择两个 task_input 进行运算, 针对操作中的每个操作: # 移除所选两个后剩余的
            task_input

            remaining_nums = [对于范围 (len(nums)) 中的每个 k, 如果
            k != j] k != i 且 结果 = evaluate_expression(nums[i], 操作, nums[j])
            如果结果不为空: # 用找到的运算结果递归检查剩余的 task_input, 表达式 =
            check_combinations ([结果] + remaining_nums) 如果找到: # 如果找到解, 则返回表达式
            返回 True, f'({nums[i]} {op} {nums[j]})' + 表达式

    return False, ""

# 尝试 task_input 的所有排列, 对于 num_permutation 在 permutations(task_input): 中找到, 表达
式 = check_combinations(list(num_permutation)) 如果找到: 返回 表达式.strip()

返回“无解”
```

清单 7：哥德尔智能体在多步数学推理任务中第 6 次迭代时发现的策略。

```
def solver(agent, task: str):  
    def parse_problem(task): # 基于关键词进行基础算术与逻辑解析  
        words = task.split()  
        numbers = list(map(int, filter(lambda x: x.isdigit(), words)))  
        return {'numbers': numbers, 'text': task}  
  
    def perform_logic_deduction(parsed_details): # 根据常见问题格式进行推理  
        numbers = parsed_details['数字'] # 这仅处理简单的加法、减法、乘法推理  
        logic_map = {  
            '加': lambda a, b: a + b, '减': lambda a, b: a - b, '乘': lambda a, b: a * b  
        } # 尝试根据关键词识别操作  
        if '和' in parsed_details['文本'] or '总计' in parsed_details['文本']:  
            结果 = 求和(数字)  
        elif '差' in parsed_details['文本'] or '少' in parsed_details['文本']:  
            结果 = logic_map['减去'](数字[0], 数字[1])  
        elif '积' in parsed_details['文本'] or '倍' in parsed_details['文本']:  
            结果 = logic_map['乘以'](数字[0], 数字[1])  
        else: # 默认情况表示无推理  
            结果 = 0  
        return 结果  
  
    def execute_computation(logic_results): # 将推理结果用于数值处理  
        return logic_results  
  
    def validate_and_compile_results(computation_results): # 准备并确保响应符合预期格式  
        final_answer = computation_results  
        return final_answer  
  
    try:  
        # 解析  
        parsed_details = parse_problem(task)  
  
        # 逻辑演绎  
        logic_results = perform_logic_deduction(parsed_details)  
  
        # 计算  
        computation_results = execute_computation(logic_results)  
  
        # 验证与编译  
        final_answer = validate_and_compile_results(computation_results)  
  
    return {"答案": final_answer} # 除非发生异常：返回 {"错误": 字符串(异常)}
```


清单 8: 哥德尔智能体在第 14 次迭代中为 MGSM 找到的策略。

```
def solver(agent, task: str): # 更新示例以反映需要分层逻辑验证的任务。examples = [ {'description': '任务示例，展示标准的基于整数的计算。', 'reasoning': 'Use arithmetic transformations to validate expressions and correct errors if any arise, ensuring correctness.', 'answer': 20}, {'description': '用于验证文字题转换为数学的示例。', 'reasoning': '从文字到数学运算的逐步解释，并桥接需要捕获的逻辑错误。', 'answer': 15}, {'description': '涉及规范化不均匀除法实例的场景。', 'reasoning': 'Ensure no division by zero and equal verification of logical conclusions.', 'answer': 6}, ]

# 任务提示结合角色，在操作结束后增强检查清单。task_prompt = "您正在引导我们作为解决方案审计员，反思每个逻辑结论以防止算术差异。" task_prompt += "任务" + "\nReflect 通过已验证示例对指令进行说明。" task_prompt += "\nExample 洞见:\n" task_prompt += ';'.join([f"示例 ['描述'] -> 推理: {示例 ['推理']} | 答案: {示例 ['答案']}" for 示例 in 示例列表]) task_prompt += "\nEnsure 计算后通过角色切换检查进行实时验证。"

消息 = [{"角色": "用户", "内容": task_prompt}]

响应 = agent.action_call_json_format_llm( model="gpt-3.5-turbo", messages=messages, 温度=0.3, num_of_response=1, role="solution 审计员", return_dict_keys=["description", "推理", "答案"], 要求=( "1. 验证计算中的算术一致性和完整性。" "2. 利用任何修正来逐步完善答案输出。" ).strip(),)

return_dict = 响应[0] 返回字典 ["答案"] = str(return_dict.get("answer", "")) 返回 return_dict
```

清单 9: 哥德尔智能体为 MGSM 找到的第 20 次迭代策略。

```
def 求解器(智能体, 任务: str): # 针对先前失误的特定易错区域进行目标设计。示例 = [ {'描述': '即时算术运算', '推理': '通过一系列逻辑确认逐步观察。', '答案': 20}, {'描述': '顺序文字题分解', '推理': '确保较小的模块段步骤与逻辑数学输出一致匹配。', '答案': 15}, {'描述': '分数与级联运算', '推理': '在进行其他形式之前验证每个分数转换。', '答案': 6}, ]

# 构建防兔语法战术线, 从角色编码检查中推动一致努力。task_prompt = "您正在通过解决方案文档推进, 重点是定位逻辑错误。" task_prompt += 任务 + "\nReflecting 通过示例验证的结构完整性。" task_prompt += "\nIn 此构造:\n" task_prompt += ';'.join([f'示例 [{描述}] -> 理由: {示例 [{推理}]} | 响应: {示例 [{答案}]} ' for 示例 in 示例 ]) task_prompt += "\nHandle 验证和清理例程, 其中错误在镜像基于行的检查中回响。"

消息 = [{"角色": "用户", "内容": task_prompt}]

响应 = agent.action_call_json_format_llm( model="gpt-3.5-涡轮", messages=messages, 温度=0.3, num_of_response=1, role="logical 验证器", return_dict_keys=["description", "推理", "答案"], 要求=( "1. 审查算术会话以全面修复错误。" "2. 以对真实语义指令流的坚定关注来发展清晰度。" ).strip(), )

return_dict = 响应[0] 返回字典 [{"答案"}] = str(return_dict.get("answer", "")) 返回
return_dict
```

Analysis Example of Gödel Agent

智能体模块中求解器当前的逻辑在验证集上仅达到 45% 的准确率。求解器的结构涉及对 LLM 的单次调用，并规定答案必须为整数。虽然这种方法看似正确，但可能不足以解决需要多部分推理或验证的更复杂问题，从而导致重大错误。已识别的问题：

1. 整数转换误解：部分数学问题并非天然产生整数答案，当求解器强制采用整数格式时，会影响准确率。
2. 推理疏漏：被指定为“数学专家”的大语言模型偶尔会遗漏必要的推理步骤或执行错误运算，导致答案错误（如样本 10）。
3. 大语言模型输出的歧义性：由于任务以多种语言给出，数学翻译或表述中可能存在歧义，影响理解（如样本 6）。

改进计划：

1. 细化推理：考虑采用多步大语言模型调用，在最终确定答案前进行推理审查。
2. 智能答案解析：不强制转换为整数，而是智能解析大语言模型的响应，判断答案是否支持整数舍入或修改。
3. 多语言处理：创建预处理步骤，将非英语数学问题统一翻译为英语，以便进行一致处理。

4. **Examples & Feedback:** Utilize correctly solved examples as context if a single-step LLM fails.

Next Steps:

1. 增强求解器逻辑：实施多步流程，包括临时大语言模型验证或推理交叉检查。
2. **Pre-process Language Variations:** Ensure all non-English tasks are translated and re-assessed.
3. 纳入示例反馈：利用评估中正确解答的示例作为参考，以引导大语言模型的推理与答案验证。